

DatawiseAgent

复现改进：任务适配、长交互和资源预算可控

汇报人：於之翔，组员：冉子易、张恒

2026.6.10

复现以后，三条值得优化的线

方向	核心问题
1. 任务适配	不同 benchmark 的失败源并不一致。InfiAgentBench 主要暴露答案名、统计口径、预处理顺序和最终格式不稳定；DataModeling 则更多暴露 submission 合法但预测低信息、metric 和 manifest 不完整等问题。因此 harness 不能只做统一 prompt，而要先识别任务类型和错误源。
2. 结构化时间换性能	长交互只有在每一轮都有明确检查目标时才有意义。我们需要把 planning、执行、debug、reformat 拆成可验证环节，失败时只修失败点，并用 final block / verifier 避免从长日志中漏抽或错抽答案。
3. 资源感知调度	强模型并不一定更慢，小模型也不一定更省。资源优化的核心是把确定性 schema、contract、format checker 前置，把强模型留给高风险推理和复杂代码生成，同时减少重复探索和无效对话。

Takeaway: 三条线不是堆模块：先识别任务错法，再把交互结构化，最后按风险决定模型、记忆和预算。

Part I

Task-Specific Adaptation

不同任务错法不同，harness 系统根据具体问题设计

Part I Baseline: 两个 benchmark 的错法不一样

InfAgentBench baseline / strong reference

Setting	ABQ	PASQ	UASQ
Qwen3-32B full latest	81.74%	86.21%	86.43%
Qwen3.5-35B-A3B baseline	86.38%	90.16%	90.35%

DataModeling baseline / strong reference

Run	Complete	Perf.	Avg time
qwen25	68/74	0.4290	760.25s
qwen35b-a3b-full	73/74	0.5318	288.60s

共同结论

通过强模型，了解答案正确率上限，同时也说明好的理解和规划可以在正确率及耗时上获得高效提升。我们的优化目标就是通过整合系统、记忆等 harness 技巧，让弱模型也能实现强模型的正确率。但针对不同任务，我们需要分别分析他们的错误源。

Part I Analysis: 任务分类分层路由解决口径漂移

核心问题

代码能跑，
但答案口径不稳定

同一张表、同一个问题，模型可能在过滤条件、统计方法、小数位或最终格式上发生轻微漂移；这些错误不一定报错，但会直接影响闭式评分。

Task-Aware harness

1. 先用 **level / concepts / keywords / format** 识别题型；
2. 不同题型路由到对应 **Harness Protocol**；
3. protocol 约束执行，并要求可复现证据；
4. verifier 检查输出是否符合任务契约。

answer name

filter

rounding

statistical method

final format

Takeaway: 重点不是让模型“知道答案”，而是把题目口径显式化为 execution contract：任务决定协议，协议约束执行，验证决定能否输出。

Part I Scheme: Task Router → Harness Protocols

路由信号

- ▶ level: easy / medium / hard
- ▶ concepts: 统计、相关性、特征、ML
- ▶ question keywords
- ▶ expected format

收束目标

不是统一大 **prompt**，
而是让不同题型进入不同协议，再由 verifier 和 Final Block 收束到稳定答案通道。

Illustration Placeholder

Task Router → Protocol Cards → Verification Gate

Stats

Corr

Feature

ML

Format

Takeaway: 后续替换为 GPT-image; 核心逻辑: 分类路由 → 协议约束 → 验证收束。

Part I Scheme: DataModeling 从 protocol 到 quality-backed

基于 InfiAgentBench 已有的改动

- ▶ Contract / Profile: 固定 metric、列、行数、target;
- ▶ Task Router: tabular、text、time-series、multi-output;
- ▶ Submission Validator: schema、ID、概率范围、sample copy;
- ▶ Quality Gate: manifest、fallback、常数预测、低信息 final。

依旧存在的问题

- ▶ 结构合法但只是 fallback;
- ▶ 多分类塌缩成单类;
- ▶ 回归/概率输出全局均值;
- ▶ CSV 可评, 但没有 validation 和选择依据。

Takeaway: 这一步不替模型建模, 只把提交前能检查清楚的内容先检查清楚。

Part I Result: Task-Aware harness 带来的直接收益

Setting	Split	ABQ	PASQ	UASQ
Qwen3-32B baseline	medium	83.72%	86.19%	86.00%
Harness rerender	medium	91.95%	93.06%	93.42%
Qwen3-32B baseline	hard	66.13%	78.49%	82.99%
Harness rerender	hard	78.41%	84.75%	87.19%

medium

ABQ 提升 **8.23 个百分点**，说明格式约束和结果抽取能直接减少可避免错误。

hard

ABQ 提升 **12.28 个百分点**，但平均分支数升至 **2.97**，复杂题仍需要控制搜索成本。

答案通道

format error / reformat missing / final block missing = 0，说明结构化答案链路有效。

Part I Result: DataModeling 的结果和边界

Setting	Complete	Perf.	Avg time
qwen25 baseline	68/74	0.4290	760.25s
qwen35b-a3b-full	73/74	0.5318	288.60s
qwen3-vl-8b old harness	74/74	0.3127	799.09s
qwen3-vl-32b old harness	74/74	0.2873	297.49s

+0.1028
qwen35b vs qwen25

74/74
qwen3-vl completion

0.45–0.50
修复典型模式后的目标区间

qwen3-vl 的问题不是不能交文件，而是合法 CSV 里常有均值、常数、fallback 或 manifest 缺失。

Takeaway: DataModeling 已经证明强模型与工程约束有收益；弱模型暴露的是 completion 到 performance 的关键断点。

Part I Insight: DataModeling 不是单一难度任务

有逻辑可循的任务

Titanic、Santander 这类 schema / metric 清晰的 tabular 任务，contract 与 validator 能帮助模型走到有效路径。

低分集中模式

多分类塌缩、均值回归、文本错路由、多输出常数、时间/分组任务错当普通表格。

下一步抓手

manifest-only repair、分级 early-stop、任务族模板、低信息预测拦截。

代表复测	分数	说明
8B Titanic	0.7877	有效 tabular submission; manifest 需要短修复
32B Titanic	0.7709	大模型不自动更好，仍受流程厚度影响
32B Santander	0.8116	预测质量可用，耗时和 manifest 是主要瓶颈

Takeaway: 弱模型低分帮助定位：哪些题需轻约束，哪些题必须进入质量驱动流程。

Failure Insights: 负结果告诉我们怎么设计系统

1. Harness 要适配模型

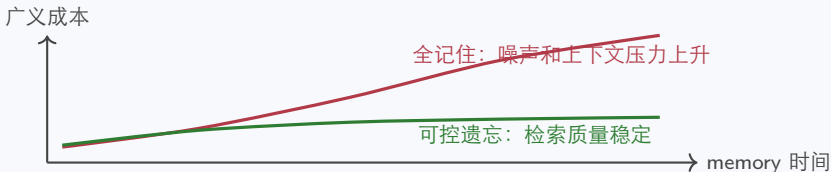
弱模型不能配过厚、过死的流程；目标是按风险调度，而不是 full-everywhere。

2. 记忆不能全记住

旧记忆、低价值记忆会降低信噪比；记忆系统需要重要性、时效性和遗忘机制。

3. Training-free 有上限

DataModeling 跨任务难度大；工程能改路径和验证，但不能完全替代强模型或后训练。



Takeaway: 失败 insight 把边界讲清楚：模型能力、记忆质量和 harness 厚度必须一起调。

Part II

Weak Model + Harness

不是多问几轮，而是把多出来的时间花在检查上

Part II Setup: 先把实验口径说清楚

Setting	实际做了什么	这组实验里怎么理解
baseline + reformat	qwen3.5-flash 按原始 DatawiseAgent 流程输出长日志, 再走原始 reformat 抽取闭式答案。	弱模型原始能力 + 原始答案抽取。
harness_light	轻量检查版: DataCard、题目要求、基础检查清单、Final Block 收束; 关闭 search、memory、heavy oracle / verifier。	目标是先修低级但致命的错: 列名、格式、小数位、答案名和最终输出通道。
harness_full	重检查版: 加入更多中间结果检查、候选检查和失败处理。	目标是继续提分, 但代价是更慢; 检查太严时也可能阻断可解析答案。
small_error_set	从错题分析里挑出的 47 道错误样本。	不是 full 257, 只能看趋势和错误类型, 不能当全量结论。

Takeaway: 先把比较对象固定住: 这里比的是同一个 47 题错题小集, 不是全量 benchmark。

Part II Result: small_error_set 上的真实结果

Setting	ABQ	Hard ABQ	Avg runtime
baseline + original reformat	48.94%	44.00%	30.24s
harness_light	59.57%	60.00%	47.03s
harness_full	63.83%	64.00%	92.79s

+10.63

light vs baseline ABQ

+16.00

light vs baseline hard ABQ

+16.79s

light runtime 增量

边界

当前只是 47 道 small_error_set。full 257 和完整消融还没验证，不能说弱模型已经接近强模型。

Takeaway: light 提分明显，额外时间还能接受；full 分数最高，但 runtime 变得很重。

Part II Analysis: 这个实验为什么有意义

弱模型主要不是“完全不会”

- ▶ 会写代码，但容易漏题目要求；
- ▶ 会算结果，但小数位、答案名、容器格式不稳；
- ▶ 会跑统计，但可能漏筛选条件或比较口径。

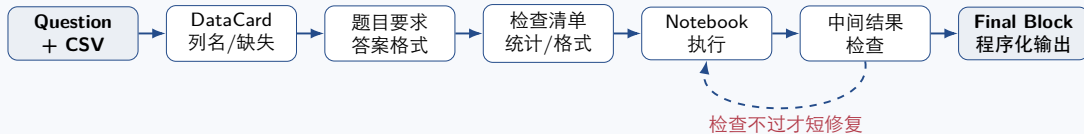
检查带来的收益和代价

- ▶ light 修了很多低级但致命的错；
- ▶ full 还能再修一点，但时间成本上升；
- ▶ 所以不是越重越好，而是看题目风险。

常见错法	轻量检查能做什么	重检查什么时候再上
格式 / 答案名错	用 Final Block 和程序化 reformat 收束。	final block 缺失或格式不合法时。
列名 / 筛选漏掉	先给 DataCard 和题目要求列表。	中间结果样本数异常时。
统计口径不稳	给相关性、异常值、预处理顺序检查清单。	hard 题、分支结果冲突或多次失败时。

Takeaway: 这个结果的价值是说明：弱模型的额外时间如果花在检查上，比单纯多问几轮更有效。

Part II Scheme: 多出来的时间具体花在哪里



程序能直接判断的

- ▶ 列是否存在，筛选后是否为空；
- ▶ final block 是否是合法 JSON；
- ▶ 小数位、列表 / 字典、@answer 是否匹配。

模型还需要判断的

- ▶ 题目到底要哪个统计口径；
- ▶ 多个候选结果哪个更可信；
- ▶ 检查失败后应修代码还是修答案。

Takeaway: 关键不是把所有事都交给模型，而是能程序检查的先程序检查，复杂判断再交给模型。

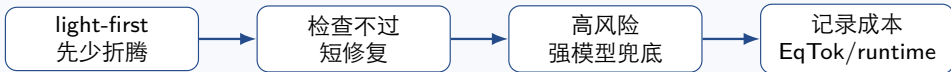
Part II Bridge: 自然引到第三部分

从 small set 得到的结论

- ▶ 弱模型加轻量检查能明显提分；
- ▶ 重检查继续提分，但 runtime 从 47.03s 到 92.79s；
- ▶ 所以下一步不是继续加模块，而是判断什么时候值得多花时间。

第三部分要回答

- ▶ 简单步骤能不能规则化？
- ▶ 明确子任务能不能交给小模型？
- ▶ 复杂规划、候选选择、失败兜底何时上强 thinking 模型？
- ▶ sub-agent 通信会不会抵消收益？



Takeaway: Part II 讲清楚 “时间花在哪里”；Part III 继续讨论 “由谁来花、哪些调用可以省”。

Part III

Resource-Aware Plan

把时间分给合适的模型；能不问模型就不问

Part III Boundary: 这部分先作为后续方案

先说边界

- ▶ resource-aware / sub-agent 还没完整验证；
- ▶ 这里主要讲后续实验设计；
- ▶ 指标用 EqTok、调用次数、runtime 一起看。

等效 token

$$\text{EqTok} = T_{32B} + 0.5T_{14B} + 0.25T_{8B}$$

把 32B / 14B / 8B 的消耗放到同一把尺上；路由收益仍是 expected / 待验证。

参考线 / 预期项	ABQ	EqTok	状态
Qwen3-32B full baseline	81.74%	5.961M	参考线
小模型路由	80.5–82.5%	约 2.120M	expected
路由 + 减少对话	81.0–83.5%	1.05–1.60M	target / 待验证

Takeaway: 这页不是说已经省下成本，而是给后续实验一个清楚的资源账本。

Part III Plan: 资源优化的两个抓手

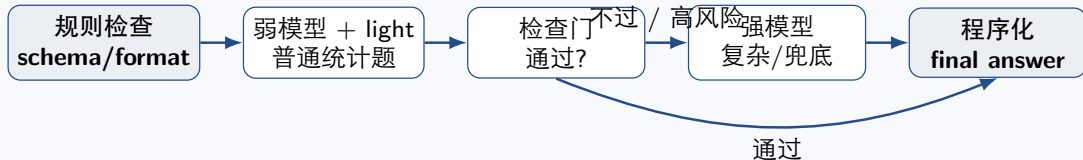
抓手	做法	答辩时怎么解释
换小模型 / 规则	固定数据检查、答案格式、列名匹配、final answer 渲染尽量程序化；普通统计题走弱模型 + light。	输入短、标准明确，不需要强模型全流程参与。
直接省掉调用	同一 CSV 的 DataCard 复用；低风险题不反复 planning；检查通过就不再回传完整日志。	省掉重复确认和长日志回传，不省必要推理。
失败后再升级	多次检查不过、候选冲突、复杂统计口径不确定时，再交给强 thinking 模型。	32B-thinking 可以做主 Agent，但全流程使用成本高。

一句话回答 planning 问题

planning 仍然重要；我们只是把 schema、格式渲染、列名检查、固定统计这类不需要重新规划的步骤拿出来，减少无意义 planning 调用。

Takeaway: 总方案不是“换成小模型”，而是少让大模型做无必要的事。

Part III Path: 可能的执行路径



强模型用在哪

复杂规划、候选选择、失败兜底；不是所有格式和固定检查都交给它。

sub-agent 怎么拆

只拆短、局部、结构化任务；否则通信轮次会抵消收益。

怎么验证

记录 EqTok、调用次数、runtime、ABQ；先小集，再 full 257。

Takeaway: 第三部分的重点是后续实验路线：简单题少折腾，难题再多花时间。

Part III Expected: 下一步怎么跑才有说服力

实验	要回答的问题	状态
full 257 baseline vs harness_light small set	提升是否能外推到全量。	待验证
最小消融	final block、检查清单、重检查分别贡献多少。	待验证
EqTok token meter	成本是否真的下降, 还是只把调用拆碎。	待实现 / 待验证
model routing	什么时候用弱模型, 什么时候强模型兜底。	后续方案

不要说满

resource-aware、sub-agent、full 257、完整消融目前都不能当已完成结论；最多作为下一步路线和预期目标。

Takeaway: 第三部分最稳的讲法：我们已经看到检查能提分，下一步要证明怎么少花冤枉钱。

Takeaway

What We Can Safely Claim

已经看到的现象 + 下一步要补的实验

最后怎么收束：哪些已经能说，哪些还不能

能说的

- ▶ InfiAgentBench 错题小集上，light harness 明显提升；
- ▶ full 版本分数更高，但时间成本也明显更高；
- ▶ DataModeling 初步看，协议错误比单纯模型能力更突出。

不能说满的

- ▶ full 257 还没有系统结论；
- ▶ resource-aware 和 sub-agent 还没有完整实验；
- ▶ DataModeling 还只是 protocol-aware 的动机，不是稳定提分验证。

Takeaway：当前最稳的结论是：轻量检查能帮助弱模型，但继续加重检查必须看时间成本和失败风险。

下一步：简单步骤程序化，困难步骤再升级

实验优先级

1. qwen3.5-flash full 257: baseline vs light;
2. 最小消融: no finalizer / no skills / no heavy check;
3. 加 token meter: EqTok、调用次数、runtime;
4. 再做 model routing / fallback。

汇报主线

- ▶ 简单题少折腾;
- ▶ 中等题用弱模型 + light 检查;
- ▶ 难题、失败题再多花时间;
- ▶ 强模型保留给复杂规划、候选选择和兜底。

Takeaway: 我们不是说已经完成一个最优系统，而是把“长交互为什么有用”拆成了可以继续验证的实验问题。

感谢聆听